Cheat sheet

Deep Learning

Neural networks, training, architectures and PyTorch

Building blocks

Neuron	`σ(w·x + b)`
Activations	ReLU, GELU, SwiGLU (transformers), sigmoid, tanh, softmax
Losses	MSE, binary cross-entropy, cross-entropy, contrastive (embeddings)
Backprop	Chain rule from loss to every weight
Optimisers	SGD (+momentum), Adam, AdamW (decoupled weight decay)
LR schedule	Warmup then cosine / linear decay

Training stability

Normalisation	BatchNorm (CNNs), LayerNorm / RMSNorm (transformers)
Residual connections	`x + f(x)` lets gradients flow in deep nets
Regularisation	Dropout, weight decay, augmentation, early stopping
Gradient clipping	Prevent exploding gradients
Mixed precision	fp16/bf16 for speed and memory

Architectures

MLP	Stacked fully-connected layers
CNN	Convolutions + pooling; images
RNN / LSTM / GRU	Sequences with recurrence; gates fight vanishing gradients
Transformer	Attention + feed-forward + residual + norm
Autoencoder / VAE	Compress and reconstruct; latent space

GAN	Generator vs discriminator
Diffusion	Learn to denoise; generate from noise

PyTorch loop

```python
model.train()
for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    loss = loss_fn(model(xb), yb)
    opt.zero_grad(); loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); sched.step()
model.eval()
with torch.no_grad(): preds = model(x_val)
```